

Structured Data Pipeline Deployment Manual

1. Purpose of This Manual

This manual provides a complete, reproducible deployment guide for the structured-data pipeline developed for this project.

It explains how to collect, ingest, process, store, and analyze the real-estate, rental, commercial sales, and population datasets that form the basis of the Structured Gentrification Index (SGI).

All instructions are designed so that any user—on any Hadoop + Hive + Spark environment—can reproduce the system end-to-end with identical outputs.

2. System Overview

The structured-data pipeline standardizes four heterogeneous datasets into a unified analytical format and computes monthly indicators that quantify socioeconomic change. The system is organized into three layers:

- **Raw Layer** — OpenAPI-based daily ingestion and historical CSV ingestion
- **Processed Layer** — Spark-based ETL and Hive external table creation
- **Analysis Layer** — Monthly feature extraction and SGI computation

Each transformation step is deterministic, ensuring reproducibility and consistency across environments.

3. Pipeline Architecture

Raw Layer → Processed Layer → Analysis Layer → SGI Output

- **Raw Layer:** Stores unmodified OpenAPI responses and historical CSV files.
- **Processed Layer:** Performs cleaning, filtering, and schema normalization via Spark ETL.
- **Analysis Layer:** Merges datasets into a unified time series, computes Month-over-Month indicators, and outputs SGI scores.

This layered design separates ingestion, transformation, and analysis, improving traceability and maintainability.

4. System Requirements

A reproducible environment requires:

- **Hadoop + HDFS (Safe Mode OFF)**

hdfs dfsadmin -safemode leave

- **Hive** with a functional Metastore and access through Beeline
 - **Spark 1.x** for Processed Layer (RDD-based ETL)
 - **Spark 3.x–4.x** for Analysis Layer (DataFrame-based computation)
 - **jq** for JSON-to-CSV transformation used in the OpenAPI collector
 - **SSH access** if deploying ETL scripts to a remote VM
-

5. Data Layer Architecture

5.1 Raw Layer

The Raw Layer maintains the exact original structure of all input data.

Two independent ingestion pathways are supported:

1. **Daily OpenAPI Pipeline (automated):** Collects previous-day data and produces raw CSVs.
2. **Historical CSV Loader (manual):** Uploads historical datasets downloaded from the Seoul Open Data Plaza.

This separation enables both retrospective and ongoing data accumulation.

5.2 Processed Layer

Spark ETL scripts transform raw datasets by:

- Cleaning malformed values
- Parsing dates into standardized fields
- Normalizing region codes (legal dong / administrative dong)
- Selecting only analysis-relevant variables

Outputs are written to:

`/user/cloudera/processed/`

Hive external tables are automatically created and mapped to these directories, enabling SQL-based inspection and downstream processing.

5.3 Analysis Layer

The Analysis Layer uses PySpark to compute the five structural indicators:

- **price_change_rate**
- **rent_change_rate**
- **sales_growth_rate**
- **youth_inflow_rate**
- **senior_outflow_rate**

All datasets are aligned to a unified **YYYYMM** time identifier, and missing months are filled using a complete region–month grid.

The SGI is produced via:

1. Z-score normalization
2. Min–max scaling to 0–100

The resulting index reflects temporal socioeconomic shifts across the target regions.

6. Script Descriptions

RawLayer

data.sh

- Collects previous-day data from the OpenAPI
- Converts JSON → CSV
- Stores raw CSV files locally

raw_upload_data.sh

- Uploads historical CSVs obtained from the Seoul Open Data Plaza
- Creates necessary RAW directories in HDFS

ProcessedLayer

run_processed.sh

- Uploads ETL scripts to the VM (if required)
- Executes Spark-submit for all four datasets:
real_estate, rent, sales, population

hive_processed.sh

- Drops and recreates all Hive external tables for the processed outputs

processed/*.py (Spark ETL scripts)

- Normalize region codes
- Parse date fields
- Filter target regions
- Output cleaned CSVs to HDFS

AnalysisLayer

analysis.py

- Builds a complete region-month grid
- Computes Month-over-Month indicators using Spark Window functions
- Produces SGI (0–100) for each region per month

7. Deployment Instructions

Historical Pipeline (manual, one-time)

bash RawLayer/raw_upload_data.sh

bash ProcessedLayer/run_processed.sh

bash ProcessedLayer/hive_processed.sh

Daily Pipeline (automated or manual)

bash RawLayer/data.sh

bash RawLayer/raw_upload_data.sh (if new historical data is added)

bash ProcessedLayer/run_processed.sh

bash ProcessedLayer/hive_processed.sh

spark-submit AnalysisLayer/analysis.py

SGI Computation Only

spark-submit AnalysisLayer/analysis.py

8. Automation Configuration

A cron job automates daily ingestion and processing.

run at 00:10 daily:

*10 0 * * * /bin/bash /home/training/DataPipeline/pipeline.sh*

pipeline.sh executes:

1. data.sh
2. raw_upload_data.sh
3. run_processed.sh
4. hive_processed.sh
5. analysis.py

This automation is essential because the OpenAPI retains only **the most recent 3 months** of records.

9. Reproducibility Assurance

The pipeline ensures full reproducibility through:

- Exact preservation of raw data

- Deterministic transformations
- No randomness in SGI calculations
- Stable and transparent computation steps
- Independence from machine-specific state

Any environment following this manual will yield identical outputs.

10. Pipeline Diagram

Raw → HDFS/raw → Spark ETL → HDFS/processed → Hive → Analysis → SGI

Final Output Directory:

final_output_zscore/
— *11300/*
— *11400/*
— *(one per region)*